Reviewer Pack — Möbius Defect of NW Design Lattice Bounds Parity Bias

Entry b27f028c321c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 10:48:15 UTC

Möbius Defect of NW Design Lattice Bounds Parity Bias

Entry ID: `b27f028c321c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-28 10:48:15 UTC

1. Conjecture

Field A (mathematical branch): Möbius theory of finite lattices (Rota–Stanley combinatorial Möbius inversion on intersection lattices; Stanley EC1 Ch.3) — rarely deployed in PRG bias analysis **Field B** (complexity object): Output bias of the Nisan–Wigderson pseudorandom generator NW_D$^{f:\{0,1\}}$d→{0,1}^m against linear (parity) distinguishers χ_T, T⊆[m]

Statement:

> Let D=(S_1,...,S_m) be an NW design over universe [d] with |S_i|=n and pairwise intersections ≤k. Let L(D) be the meet-semilattice of all distinct intersections T_I=⋂*{i∈I} S_i (I⊆[m]) ordered by ⊆ with bottom 0̂=⋂, and let μ_L be its Möbius function. Define the Möbius defect δ(D)=Σ*{T∈L(D), T≠⋂} |μ_L(0̂,T)|·2^{−|T|}. Conjecture: for every f:{0,1}^n→{0,1} with max Walsh coefficient |f̂(U)|≤2^{−n/3} on |U|≥k, and every nonempty T⊆[m], |E_{y∼U_d}[χ_T(NW_D^f(y))]| ≤ 4·δ(D).

Rationale (proposer's reasoning):

> Standard NW bias bounds use the crude factor m·2·ε from a hybrid argument that ignores higher-order inclusion–exclusion cancellation across the design's intersection pattern. Möbius values μ_L(0̂,T) are precisely the signed multiplicities by which intersections of design rows are over/under-counted, so replacing m by δ(D) should yield substantially tighter bias estimates whenever L(D) is structured (e.g., projective-plane or polynomial designs versus random subset-systems). If the conjecture holds, δ(D) becomes a computable, design-only certificate of fooling power independent of f beyond the Fourier-tail hypothesis.

Taxonomy category: `NISAN_WIGDERSON_DESIGNS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8e65bd9767ab0d14`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ≥200 NW designs (≥100 rejection-sampled + ≥100 polynomial-construction) over n∈{4..9}, k=2, d∈{8..18}, with ≥2 hard f per design satisfying the Fourier-tail hypothesis, compute exact bias E_y[χ_T(NW_D^f(y))] for every non-empty T⊆[m] of weight ≤4. SUPPORTED iff |bias|≤4·δ(D) on every (D,f,T) AND OLS slope of log|bias| vs log δ(D) ≥0.5; FALSIFIED iff any single triple violates |bias|≤4·δ(D).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Nisan-Wigderson generator parity linear distinguisher bias design intersection` - `Mobius function intersection lattice pseudorandom generator Walsh Fourier bias` - `Rota Mobius inversion combinatorial design pseudorandomness parity correlation bound`

Top relevant hits considered: - [http://arxiv.org/abs/2601.03730v1] Perception-Aware Bias Detection for Query Suggestions - [http://arxiv.org/abs/1207.0393v2] Pseudo-finite hard instances for a student-teacher game with a Nisan-Wigderson generator - [http://arxiv.org/abs/2109.14047v1] Second Order WinoBias (SoWinoBias) Test Set for Latent Gender Bias Detection in Coreference Resolution - [http://arxiv.org/abs/0710.4339v1] Heavy-Quark Masses from the Fermilab Method in Three-Flavor Lattice QCD - [http://arxiv.org/abs/1111.0981v1] Form factors for B to Kll semileptonic decay from three-flavor lattice QCD - [http://arxiv.org/abs/1504.04131v2] Formulas for the Walsh coefficients of smooth functions and their application to bounds on the Walsh coefficients - [http://arxiv.org/abs/2202.01071v5] Autocorrelation of the Mobius Function - [http://arxiv.org/abs/1409.3052v2] Rota-Baxter Coalgebras

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import random
import math
import json

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i + 1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
```

```
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        factor = -A[i][i]
        for j in range(n):
            A[i][j] /= factor
        for j in range(m):
            if j != i:
                factor = A[j][i]
                for k in range(n):
                    A[j][k] += factor * A[i][k]
    return A

def matrix_multiply(A, B):
    m, n, p = len(A), len(B[0]), len(B)
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def mobius_function(L, T):
    if not T:
        return 1
    S = [s for s in L if s.issubset(T) and len(s) == len(T) - 1]
    return sum(-mobius_function(L, s) for s in S)

def nw_design(d, n, k):
    S = []
    while len(S) < d:
        subset = set(random.sample(range(1, d + 1), n))
        if all(len(subset & s) <= k for s in S):
            S.append(subset)
    return S

def nw_design_polynomial(d, n):
    p = 2
    x = [random.randint(0, p - 1) for _ in range(n)]
    y = [random.randint(0, p - 1) for _ in range(n)]
    z = [random.randint(0, p - 1) for _ in range(n)]
    return [[x[i] * y[j] + z[k] for k in range(n)] for i in range(n)]

def walsh_transform(f):
    n = len(f)
    F = [[f[i ^ j] for j in range(1 << n)] for i in range(1 << n)]
    A = gaussian_elimination(F)
    return [sum(A[i][j] * (2 ** -n) for j in range(n)) for i in range(1 << n)]

def nw_design_function(d, n, k):
    if random.random() < 0.5:
        return nw_design(d, n, k)
    else:
        return nw_design_polynomial(d, n)

def compute_delta(D):
    L = []
```

```
    for i in range(1 << len(D)):
        T = {j + 1 for j in range(len(D)) if (i & (1 << j))}
        if all(len(T.intersection(s)) <= k for s in D):
            L.append(T)
    mu_L = {}
    for T in L:
        mu_L[T] = mobius_function(L, T)
    delta_D = sum(abs(mu_L[T]) * 2 ** -len(T) for T in L if T)
    return delta_D

def compute_bias(D, f):
    m = len(D)
    bias = [0] * (1 << m)
    for y in range(1 << m):
        NW_D_f_y = [sum(f[i] * D[y & (1 << i)] for i in range(m)) % 2 for _ in range(1 << n)]
        for T in range(1, 1 << m):
            if T.bit_count() <= 4:
                bias[T] += NW_D_f_y[random.randint(0, len(NW_D_f_y) - 1)]
    return [bias[T] / (1 << n) for T in range(1 << m)]

def is_hard_function(f, delta_D):
    return any(abs(bias) > 4 * delta_D for bias in f)

def compute_ols_slope(bias_values, delta_values):
    if not bias_values or not delta_values:
        return None
    log_bias = [math.log(abs(bias)) for bias in bias_values]
    log_delta = [math.log(delta) for delta in delta_values]
    n = len(log_bias)
    sum_log_bias = sum(log_bias)
    sum_log_delta = sum(log_delta)
    sum_log_bias_squared = sum(x * x for x in log_bias)
    sum_log_bias_log_delta = sum(x * y for x, y in zip(log_bias, log_delta))
    slope = (n * sum_log_bias_log_delta - sum_log_bias * sum_log_delta) / (n * sum_log_bias_sq
    return slope

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 8, 11, 14]
    d_values = range(8, 19)
    k = 2
    num_designs = 200
    num_hard_functions_per_design = 10

    results = []
    for n in n_values:
        for _ in range(num_designs):
            D = nw_design_function(d_values[seed % len(d_values)], n, k)
            delta_D = compute_delta(D)
            f = walsh_transform([random.randint(0, 1) for _ in range(2 ** n)])
            if is_hard_function(f, delta_D):
                bias_values = compute_bias(D, f)
                ols_slope = compute_ols_slope(bias_values, [delta_D] * len(bias_values))
                results.append({
                    "metric_name": "OLS Slope",
```

```
                        "metric_value": ols_slope,
                        "instances_tested": 1,
                        "conjecture_holds": ols_slope is not None and ols_slope >= 0.5,
                        "counterexample": "" if ols_slope is not None else "mapping_undefined"
                    })

        mean_metric = sum(result["metric_value"] for result in results) / len(results)
        std_metric = math.sqrt(sum((result["metric_value"] - mean_metric) ** 2 for result in resul
        support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(result

        return {
            "seed": seed,
            "mean_metric": mean_metric,
            "std_metric": std_metric,
            "support_fraction": support_fraction
        }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(json.dumps({"TRIAL": result}))
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
{"TRIAL": null}
{"TRIAL": null}
{"TRIAL": null}
{"TRIAL": null}
{"TRIAL": null}
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The stdout shows only {"TRIAL": null} lines with empty aggregate_stats and empty per_seed_brief — this is not evidence of SUPPORTED, it's evidence the test never produced a measurement. Even setting that aside, the Möbius defect δ(D) is a sum over a potentially exponential lattice with |μ_L| values that can be large, so for small n≤15 the bound 4·δ(D) almost certainly exceeds 1 and is vacuous (parity bias is trivially ≤1). This is a classic metric saturation / vacuous-bound failure mode combin

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test stdout contains only {"TRIAL": null} entries with no measurements, no RESULT line, and no evidence that the pre-registered criteria (≥200 designs, both constructions, regression slope ≥0.5, zero violations) were evaluated. The critic also flags a likely

vacuous-bound saturation issue, so SUPPORTED cannot be claimed. | next: Fix the test harness so trials actually emit (D,f,T,bias,δ(D)) tuples, then verify whether 4·δ(D) is non-trivial (<1) for the tested regime before re-running the p

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	122167
2	preregistration	claude_max	opus	0	0	12085
3	novelty	claude_max	opus	0	0	4033
4	novelty	claude_max	opus	0	0	8914
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16195
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17425
7	critic	claude_max	opus	0	0	11519
8	judge	claude_max	opus	0	0	5960

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 198297 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/b27f028c321c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b27f028c321c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b27f028c321c.tar.gz` (if generated)